Lesson17_GDC_Tidyverse_dplyr.Rmd

2025-05-06

Importing Example Datasets

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
clinical = read_tsv('clinical.tsv', na=c("", "NA", "'--"))
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 5546 Columns: 210
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr  (52): project.project_id, cases.case_id, cases.consent_type, cases.dise...
## dbl  (15): cases.days_to_consent, demographic.age_at_index, demographic.days...
## lgl (143): cases.days_to_lost_to_followup, demographic.age_is_obfuscated, de...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(clinical)
```

```
## # A tibble: 6 x 210
##   project.project_id cases.case_id      cases.consent_type cases.days_to_consent
##   <chr>              <chr>              <chr>                              <dbl>
## 1 TCGA-BRCA          001cef41-ff86-4d3~ Informed Consent                     -34
## 2 TCGA-BRCA          001cef41-ff86-4d3~ Informed Consent                     -34
## 3 TCGA-BRCA          001cef41-ff86-4d3~ Informed Consent                     -34
## 4 TCGA-BRCA          0045349c-69d9-430~ Informed Consent                      76
## 5 TCGA-BRCA          00807dae-9f4a-4fd~ Informed Consent                      19
## 6 TCGA-BRCA          00807dae-9f4a-4fd~ Informed Consent                      19
## # i 206 more variables: cases.days_to_lost_to_followup <lgl>,
## #   cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
```

```
## #   demographic.cause_of_death_source <lgl>,
## #   demographic.country_of_birth <lgl>, ...
```

```r
library(dplyr)
penguins = read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv")
```

```
## Rows: 344 Columns: 7
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (3): species, island, sex
## dbl (4): bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(penguins)
```

```
## # A tibble: 6 x 7
##   species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
## 1 Adelie  Torgersen           39.1          18.7               181        3750
## 2 Adelie  Torgersen           39.5          17.4               186        3800
## 3 Adelie  Torgersen           40.3          18                 195        3250
## 4 Adelie  Torgersen           NA            NA                  NA          NA
## 5 Adelie  Torgersen           36.7          19.3               193        3450
## 6 Adelie  Torgersen           39.3          20.6               190        3650
## # i 1 more variable: sex <chr>
```

Using what we learnt yesterday about data frame and accessing columns. How might we change the coding of `sex` from MALE/FEMALE in Palmers Penguins to M/F?

```r
library(stringr)
penguins$sex = str_replace_all(penguins$sex, c("^MALE"='M', "^FEMALE"='F'))
penguins
```

```
## # A tibble: 344 x 7
##    species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
##  1 Adelie  Torgersen           39.1          18.7               181        3750
##  2 Adelie  Torgersen           39.5          17.4               186        3800
##  3 Adelie  Torgersen           40.3          18                 195        3250
##  4 Adelie  Torgersen           NA            NA                  NA          NA
##  5 Adelie  Torgersen           36.7          19.3               193        3450
##  6 Adelie  Torgersen           39.3          20.6               190        3650
##  7 Adelie  Torgersen           38.9          17.8               181        3625
##  8 Adelie  Torgersen           39.2          19.6               195        4675
##  9 Adelie  Torgersen           34.1          18.1               193        3475
## 10 Adelie  Torgersen           42            20.2               190        4250
## # i 334 more rows
## # i 1 more variable: sex <chr>
```

Using Dplyr to Arrange, Manipulate, and Mutate Data Frames

dplyr provides functions to manipulate data. The common syntax of `dplyr` is:

1. First argument is always the data frame that you want to manipulate

2. Subsequent arguments describe which columns you want to manipulate and how you want to manipulate them.
3. The result will always be a new data frame

dplyr will never modify an input dataset - but will always return a new tibble data frame.

dplyr verbs are organized into 4 group based upon operating on: **rows**, **columns**, **groups**, or **tables**

Group	dplyr Function	Description
	|>, %>%	Pipe operator. Inserts the left-hand side as the first argument in the right-hand side call. Small differences exist between |> and %>%. Note: `|>` was only added in R version 4.10
	%in%	
ROW	filter()	Allows you to keep rows based on the values of a column
	arrange()	Changes the order of the rows based on the values of the columns
	distinct()	Finds all the unique rows in a dataset
	count()	Returns the count of unique items
COLUMN	mutate()	Add new columns that are calculated from the existing columns
	select()	Easily select variables by name in the dataframe
	rename()	Allows you to rename a few variables
	relocate()	Move variables around
GROUP	group_by()	Divide your dataset into groups meaningful for your analysis
	summarize()	Calculate a single summary statistic, reduces the data frame to have a single row for each group
	slice_	Allows you extract specific rows within each group • `slice_head(n=n)` Takes the first n rows from each group • `slice_tail(n=n)` Takes the last n rows from each group • `slice_min(x, n=n)` Takes n rows with the smallest value of column x • `slice_max(x, n=n)` Takes n rows with the largest value of columns x • `slice_sample(n=n)` Gets n random rows
	.by	Group arguments and works with all previous functions()
	ungroup()	Remove grouping from data frame
TABLE	_join()	Combine multiple datasets X and Y using _join functionality. See https://r4ds.hadley.nz/joins for examples and information about performing: • left_join() - Keeps observations in X - adds columns to current rows • inner_join() - Only keeps observations from X that have matching key in Y • right_join() - Only keeps observations in Y • full_join() - Keeps all observations in X and Y (adds columns and rows) • semi_join() - Returns rows in X that have a match in Y • anti_join() - Returns rows in X that don't have a match in Y

There are some small differences between `|>` and `%>%`. The `|>` is the simpler of the two, while `%>%` provides additional options. https://www.tidyverse.org/blog/2023/04/base-vs-magrittr-pipe/

If we use the pipe `|>` argument, the first argument (table variable) is filled in automatically.

Note: `|>` was introduced in 4.1, and is equivalent to `%>%`. You'll see many older tutorials using the old `%>%` operator instead of the newer `|>` R native pipe operator.

Row Functions

distinct() `distinct()` will return unique rows in the dataset. If you supply a single column it'll return the unique column entries. If you supply multiple columns, it'll find only the unique column pairs. If you want to keep the other columns, you can add the argument `.keep_all=TRUE`

```
penguins |> distinct(species, island)
```

```
## # A tibble: 5 x 2
##   species   island
##   <chr>     <chr>
## 1 Adelie    Torgersen
## 2 Adelie    Biscoe
## 3 Adelie    Dream
## 4 Chinstrap Dream
## 5 Gentoo    Biscoe
```

```
penguins |> distinct(species, island, .keep_all=TRUE)
```

```
## # A tibble: 5 x 7
##   species   island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <chr>     <chr>              <dbl>         <dbl>             <dbl>       <dbl>
## 1 Adelie    Torgersen           39.1          18.7               181        3750
## 2 Adelie    Biscoe              37.8          18.3               174        3400
## 3 Adelie    Dream               39.5          16.7               178        3250
## 4 Chinstrap Dream               46.5          17.9               192        3500
## 5 Gentoo    Biscoe              46.1          13.2               211        4500
## # i 1 more variable: sex <chr>
```

```
clinical |> distinct(cases.case_id)
```

```
## # A tibble: 1,098 x 1
##    cases.case_id
##    <chr>
##  1 001cef41-ff86-4d3f-a140-a647ac4b10a1
##  2 0045349c-69d9-4306-a403-c9c1fa836644
##  3 00807dae-9f4a-4fd1-aac2-82eb11bf2afb
##  4 00a2d166-78c9-4687-a195-3d6315c27574
##  5 00b11ca8-8540-4a3d-b602-ec754b00230b
##  6 011b9b2d-ebe5-42bf-9662-d922faccc7a1
##  7 01263518-5f7c-49dc-8d7e-84b0c03a6a63
##  8 0130d616-885e-4a6c-9d03-2f17dd692a05
##  9 01674b2c-5cf2-478f-84a1-f69c39f47bd4
## 10 016caf42-4e19-4444-ab5d-6cf1e76c4afa
## # i 1,088 more rows
```

```
(clinical_uniq = clinical |> distinct(cases.case_id, .keep_all=TRUE))
```

```
## # A tibble: 1,098 x 210
##    project.project_id cases.case_id     cases.consent_type cases.days_to_consent
##    <chr>              <chr>             <chr>                              <dbl>
##  1 TCGA-BRCA          001cef41-ff86-4d~ Informed Consent                     -34
##  2 TCGA-BRCA          0045349c-69d9-43~ Informed Consent                      76
##  3 TCGA-BRCA          00807dae-9f4a-4f~ Informed Consent                      19
##  4 TCGA-BRCA          00a2d166-78c9-46~ Informed Consent                       0
##  5 TCGA-BRCA          00b11ca8-8540-4a~ Informed Consent                      11
##  6 TCGA-BRCA          011b9b2d-ebe5-42~ Informed Consent                      16
```

```
##  7 TCGA-BRCA          01263518-5f7c-49~ Informed Consent                      31
##  8 TCGA-BRCA          0130d616-885e-4a~ Informed Consent                      36
##  9 TCGA-BRCA          01674b2c-5cf2-47~ Informed Consent                      13
## 10 TCGA-BRCA          016caf42-4e19-44~ Informed Consent                       5
## # i 1,088 more rows
## # i 206 more variables: cases.days_to_lost_to_followup <lgl>,
## #   cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
## #   demographic.cause_of_death_source <lgl>, ...
```

---

filter(df, conditions . . .)

Operator	Description
<, <=, >, >=, ==	Mathematical comparison operators
|	OR
&	AND
xor()	Exclusive OR (Excludes things in both)
%in%	Checks left value/argument is in the right value/argument
is.na(), !is.na()	Is NA or is not NA()
!	Negates expression

The `condition...` can be one or more conditional statements that evaluate the values in one or more columns. The output from `filter()` is a new `tibble` data frame.

```
adelie = penguins |> filter(species=='Adelie' & island=='Dream' & sex=='MALE' & body_mass_g>3000)
adelie
```

```
## # A tibble: 0 x 7
## # i 7 variables: species <chr>, island <chr>, bill_length_mm <dbl>,
## #   bill_depth_mm <dbl>, flipper_length_mm <dbl>, body_mass_g <dbl>, sex <chr>
```

The conditionals can be combined using the logical operator `&` (AND) or `|` (OR). If you separate the conditions using commas, `filter()` will interpret them as AND operators.

In addition to the standard comparison operators, **dplyr** accepts the `%in%` operator, which allows you to search from a list or vector of items.

```
adelie = penguins |> filter(species=='Adelie', island %in% c('Dream','Biscoe'), sex=='MALE')
adelie
```

```
## # A tibble: 0 x 7
## # i 7 variables: species <chr>, island <chr>, bill_length_mm <dbl>,
## #   bill_depth_mm <dbl>, flipper_length_mm <dbl>, body_mass_g <dbl>, sex <chr>
```

```
(male = clinical_uniq |> filter(demographic.gender=='male'))
```

```
## # A tibble: 12 x 210
##    project.project_id cases.case_id     cases.consent_type cases.days_to_consent
##    <chr>              <chr>             <chr>                              <dbl>
##  1 TCGA-BRCA          1026e0bf-5640-4f~ Informed Consent                      50
##  2 TCGA-BRCA          35ca5e2a-861a-4d~ Informed Consent                      15
##  3 TCGA-BRCA          3c08aabd-d5b5-4b~ Informed Consent                     394
```

```
##  4 TCGA-BRCA          4dfc233b-ed03-48~ Informed Consent                      61
##  5 TCGA-BRCA          784de7ac-8424-42~ Informed Consent                    -365
##  6 TCGA-BRCA          aba5f46a-e67a-4c~ Informed Consent                      42
##  7 TCGA-BRCA          c710f4ab-fbd9-42~ Informed Consent                      13
##  8 TCGA-BRCA          d2219a3f-4c55-4d~ Informed Consent                       0
##  9 TCGA-BRCA          d5f2b85a-94a9-41~ Informed Consent                       2
## 10 TCGA-BRCA          daf62b56-6dd2-4a~ Informed Consent                      34
## 11 TCGA-BRCA          dba9beec-5e51-48~ Informed Consent                       6
## 12 TCGA-BRCA          eadb0562-2165-4a~ Informed Consent                      -1
## # i 206 more variables: cases.days_to_lost_to_followup <lgl>,
## #   cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
## #   demographic.cause_of_death_source <lgl>,
## #   demographic.country_of_birth <lgl>, ...
```

---

arrange() `arrange()` sorts the rows based on the values in a column(s). By providing multiple columns you can perform more complicated sorting. If you want to sort a column in descending order, you surround the column name with `desc()` function.

```r
(adelie_sorted = adelie |> arrange(body_mass_g, desc(bill_depth_mm)))
```

```
## # A tibble: 0 x 7
## # i 7 variables: species <chr>, island <chr>, bill_length_mm <dbl>,
## #   bill_depth_mm <dbl>, flipper_length_mm <dbl>, body_mass_g <dbl>, sex <chr>
```

---

count() `count()` will return the number of unique rows or columns pairs as a new column **n**

```r
adelie |> count(species, island)
```

```
## # A tibble: 0 x 3
## # i 3 variables: species <chr>, island <chr>, n <int>
```

```r
(clinical_uniq |> count(demographic.race))
```

```
## # A tibble: 6 x 2
##   demographic.race                     n
##   <chr>                            <int>
## 1 american indian or alaska native     1
## 2 asian                               61
## 3 black or african american          183
## 4 not reported                        95
## 5 white                              757
## 6 <NA>                                 1
```

Column Functions

mutate() - create a new column based upon the values of the other columns `mutate()` adds one or more new columns to the the right-hand side of the dataset.

```r
penguins |> mutate(weight_lbs = body_mass_g/454,
                   weight_kg = body_mass_g/1000
                   )
```

```
## # A tibble: 344 x 9
##    species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
##  1 Adelie  Torgersen           39.1          18.7               181        3750
##  2 Adelie  Torgersen           39.5          17.4               186        3800
##  3 Adelie  Torgersen           40.3          18                 195        3250
##  4 Adelie  Torgersen           NA            NA                  NA          NA
##  5 Adelie  Torgersen           36.7          19.3               193        3450
##  6 Adelie  Torgersen           39.3          20.6               190        3650
##  7 Adelie  Torgersen           38.9          17.8               181        3625
##  8 Adelie  Torgersen           39.2          19.6               195        4675
##  9 Adelie  Torgersen           34.1          18.1               193        3475
## 10 Adelie  Torgersen           42            20.2               190        4250
## # i 334 more rows
## # i 3 more variables: sex <chr>, weight_lbs <dbl>, weight_kg <dbl>
```

You can modify the placement of the new columns using the `.before=` or `.after` arguments in mutate. They will either take the indices of the columns, or columns name.

```r
penguins |> mutate(weight_lbs = body_mass_g/454,
                   weight_kg = body_mass_g/1000,
                   .after=body_mass_g
                   )
```

```
## # A tibble: 344 x 9
##    species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
##  1 Adelie  Torgersen           39.1          18.7               181        3750
##  2 Adelie  Torgersen           39.5          17.4               186        3800
##  3 Adelie  Torgersen           40.3          18                 195        3250
##  4 Adelie  Torgersen           NA            NA                  NA          NA
##  5 Adelie  Torgersen           36.7          19.3               193        3450
##  6 Adelie  Torgersen           39.3          20.6               190        3650
##  7 Adelie  Torgersen           38.9          17.8               181        3625
##  8 Adelie  Torgersen           39.2          19.6               195        4675
##  9 Adelie  Torgersen           34.1          18.1               193        3475
## 10 Adelie  Torgersen           42            20.2               190        4250
## # i 334 more rows
## # i 3 more variables: weight_lbs <dbl>, weight_kg <dbl>, sex <chr>
```

You select a subset of columns to be kept in the output dataset using the `.keep` argument. The `.keep` argument will take the following options

- `"all"` retains all columns from `.data`. This is the default.
- `"used"` retains only the columns used in `...` to create new columns. This is useful for checking your work, as it displays inputs and outputs side-by-side.
- `"unused"` retains only the columns *not* used in `...` to create new columns. This is useful if you generate new columns, but no longer need the columns used to generate them.
- `"none"` doesn't retain any extra columns from `.data`. Only the grouping variables and columns created by `...` are kept.

```r
penguins |> mutate(weight_lbs = body_mass_g/454,
                   weight_kg = body_mass_g/1000,
                   .keep="used"
                   )
```

```
## # A tibble: 344 x 3
##    body_mass_g weight_lbs weight_kg
##          <dbl>      <dbl>     <dbl>
##  1        3750       8.26      3.75
##  2        3800       8.37      3.8
##  3        3250       7.16      3.25
##  4          NA      NA        NA
##  5        3450       7.60      3.45
##  6        3650       8.04      3.65
##  7        3625       7.98      3.62
##  8        4675      10.3       4.68
##  9        3475       7.65      3.48
## 10        4250       9.36      4.25
## # i 334 more rows
```

```r
penguins |> select(species, island, sex)
```

select() - subset data frame based on column names or matches

```
## # A tibble: 344 x 3
##    species island    sex
##    <chr>   <chr>     <chr>
##  1 Adelie  Torgersen M
##  2 Adelie  Torgersen F
##  3 Adelie  Torgersen F
##  4 Adelie  Torgersen <NA>
##  5 Adelie  Torgersen F
##  6 Adelie  Torgersen M
##  7 Adelie  Torgersen F
##  8 Adelie  Torgersen M
##  9 Adelie  Torgersen <NA>
## 10 Adelie  Torgersen <NA>
## # i 334 more rows
```

You can also use the `:` operator to select a set of columns `from:to` . By prefixing the `from:to` with an exclamation point, like `!from:to` you can specify that you want to exclude those columns.

```r
penguins |> select(!bill_length_mm:flipper_length_mm)
```

```
## # A tibble: 344 x 4
##    species island    body_mass_g sex
##    <chr>   <chr>           <dbl> <chr>
##  1 Adelie  Torgersen        3750 M
##  2 Adelie  Torgersen        3800 F
##  3 Adelie  Torgersen        3250 F
##  4 Adelie  Torgersen          NA <NA>
##  5 Adelie  Torgersen        3450 F
##  6 Adelie  Torgersen        3650 M
##  7 Adelie  Torgersen        3625 F
##  8 Adelie  Torgersen        4675 M
```

```
##  9 Adelie  Torgersen        3475 <NA>
## 10 Adelie  Torgersen        4250 <NA>
## # i 334 more rows
```

Using `where()` function within the `select()` you can select columns based on their attributes, such as character datatypes.

Only select the columns that are *character* datatypes

```r
penguins |> select(where(is.character))
```

```
## # A tibble: 344 x 3
##    species island    sex
##    <chr>   <chr>     <chr>
##  1 Adelie  Torgersen M
##  2 Adelie  Torgersen F
##  3 Adelie  Torgersen F
##  4 Adelie  Torgersen <NA>
##  5 Adelie  Torgersen F
##  6 Adelie  Torgersen M
##  7 Adelie  Torgersen F
##  8 Adelie  Torgersen M
##  9 Adelie  Torgersen <NA>
## 10 Adelie  Torgersen <NA>
## # i 334 more rows
```

Select columns that start with *cases* or *demographics*

```r
library(stringr)
(clinical_subset = clinical_uniq |> select(contains("cases.") | contains('demographic.')))
```

```
## # A tibble: 1,098 x 31
##    cases.case_id cases.consent_type cases.days_to_consent cases.days_to_lost_t~1
##    <chr>         <chr>                             <dbl> <lgl>
##  1 001cef41-ff8~ Informed Consent                    -34 NA
##  2 0045349c-69d~ Informed Consent                     76 NA
##  3 00807dae-9f4~ Informed Consent                     19 NA
##  4 00a2d166-78c~ Informed Consent                      0 NA
##  5 00b11ca8-854~ Informed Consent                     11 NA
##  6 011b9b2d-ebe~ Informed Consent                     16 NA
##  7 01263518-5f7~ Informed Consent                     31 NA
##  8 0130d616-885~ Informed Consent                     36 NA
##  9 01674b2c-5cf~ Informed Consent                     13 NA
## 10 016caf42-4e1~ Informed Consent                      5 NA
## # i 1,088 more rows
## # i abbreviated name: 1: cases.days_to_lost_to_followup
## # i 27 more variables: cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
## #   demographic.cause_of_death_source <lgl>, ...
```

Group Functions

group_by() - divide your dataset into meaningful groups `group_by()` doesn't changed the data, but will add a **Groups** attribute which shows how the data is now grouped. The power of grouping comes in

downstream functions - because you can now summarize or analyze them by the grouping.

```
penguins |> group_by(sex)
```

```
## # A tibble: 344 x 7
## # Groups:   sex [3]
##    species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
##  1 Adelie  Torgersen           39.1          18.7               181        3750
##  2 Adelie  Torgersen           39.5          17.4               186        3800
##  3 Adelie  Torgersen           40.3          18                 195        3250
##  4 Adelie  Torgersen           NA            NA                  NA          NA
##  5 Adelie  Torgersen           36.7          19.3               193        3450
##  6 Adelie  Torgersen           39.3          20.6               190        3650
##  7 Adelie  Torgersen           38.9          17.8               181        3625
##  8 Adelie  Torgersen           39.2          19.6               195        4675
##  9 Adelie  Torgersen           34.1          18.1               193        3475
## 10 Adelie  Torgersen           42            20.2               190        4250
## # i 334 more rows
## # i 1 more variable: sex <chr>
```

```
clinical_uniq |> group_by(demographic.gender)
```

```
## # A tibble: 1,098 x 210
## # Groups:   demographic.gender [3]
##    project.project_id cases.case_id     cases.consent_type cases.days_to_consent
##    <chr>              <chr>             <chr>                              <dbl>
##  1 TCGA-BRCA          001cef41-ff86-4d~ Informed Consent                     -34
##  2 TCGA-BRCA          0045349c-69d9-43~ Informed Consent                      76
##  3 TCGA-BRCA          00807dae-9f4a-4f~ Informed Consent                      19
##  4 TCGA-BRCA          00a2d166-78c9-46~ Informed Consent                       0
##  5 TCGA-BRCA          00b11ca8-8540-4a~ Informed Consent                      11
##  6 TCGA-BRCA          011b9b2d-ebe5-42~ Informed Consent                      16
##  7 TCGA-BRCA          01263518-5f7c-49~ Informed Consent                      31
##  8 TCGA-BRCA          0130d616-885e-4a~ Informed Consent                      36
##  9 TCGA-BRCA          01674b2c-5cf2-47~ Informed Consent                      13
## 10 TCGA-BRCA          016caf42-4e19-44~ Informed Consent                       5
## # i 1,088 more rows
## # i 206 more variables: cases.days_to_lost_to_followup <lgl>,
## #   cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
## #   demographic.cause_of_death_source <lgl>, ...
```

summarize(...) - Calculates a single summary statistic for each group. Resulting in a row of summary statistic for every grouping.

```
penguins |> group_by(sex) |> summarize(average_weight=mean(body_mass_g))
```

```
## # A tibble: 3 x 2
##   sex   average_weight
##   <chr>          <dbl>
## 1 F              3862.
## 2 M              4546.
## 3 <NA>             NA
```

```r
clinical_uniq |> group_by(demographic.gender) |> summarize(avg_age = mean(demographic.age_at_index))
```

```
## # A tibble: 3 x 2
##   demographic.gender avg_age
##   <chr>                <dbl>
## 1 female                58.4
## 2 male                  62.6
## 3 <NA>                  NA
```

NA's in the data may cause issues when performing analysis. If you need to remove NA values, you can add the options `na.rm=TRUE` in `mean()` and other summary statistic function to remove NA values before calculating.

Finally, `summarize()` can take multiple arguments, so you can have multiple descriptive statistics. Along with the `mean()`, we might want to additional information.

R Function	Description
min()	Minimum value
max()	Maximum value
mean()	Mean value
median()	Median value
quartile(data, fraction)	Quartile value for fraction of data: 0.25 = 1st Quartile, 0.75 = 3rd Quartile
IQR()	Interquartile Range
sd()	Standard Deviation
var()	Variance
n()	Number of entries or rows

```r
penguins |> group_by(sex) |> summarize(average_weight=mean(body_mass_g), STDEV=sd(body_mass_g), N=n())
```

```
## # A tibble: 3 x 4
##   sex   average_weight STDEV     N
##   <chr>          <dbl> <dbl> <int>
## 1 F              3862.  666.   165
## 2 M              4546.  788.   168
## 3 <NA>             NA    NA     11
```

You may get an warning message, when more than one group is being used. This is because function behaviour peels off the last grouping. See https://r4ds.hadley.nz/data-transform#grouping-by-multiple-variables for more information.

```r
penguins |> group_by(species, sex) |> summarize(average_weight=mean(body_mass_g))
```

```
## `summarise()` has grouped output by 'species'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 8 x 3
## # Groups:   species [3]
##   species   sex   average_weight
##   <chr>     <chr>          <dbl>
## 1 Adelie    F              3369.
## 2 Adelie    M              4043.
## 3 Adelie    <NA>             NA
## 4 Chinstrap F              3527.
## 5 Chinstrap M              3939.
## 6 Gentoo    F              4680.
```

```
## 7 Gentoo    M              5485.
## 8 Gentoo    <NA>             NA
```

Exercise: Find the average and max age_of_index by country_of_residence_at_enrollment and gender

Get the `mean()` and `max()` age of index for the clinical data grouping by country of residence and gender.

```
clinical_uniq |>
```

---

ungroup() - removes the grouping from the Tibble data frame If you need to remove the grouping from a data set, you can pass it to `ungroup()`. The changes will be visible by the **Groups** attribute being removed from the dataset.

```
x = penguins |> group_by(sex) |> slice_sample(prop=0.1)
head(x)
```

```
## # A tibble: 6 x 7
## # Groups:   sex [1]
##   species   island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <chr>     <chr>              <dbl>         <dbl>             <dbl>       <dbl>
## 1 Adelie    Torgersen           38.8          17.6               191        3275
## 2 Gentoo    Biscoe              45.7          13.9               214        4400
## 3 Chinstrap Dream               45.7          17.3               193        3600
## 4 Adelie    Biscoe              39.6          17.7               186        3500
## 5 Adelie    Torgersen           34.4          18.4               184        3325
## 6 Adelie    Dream               37.3          16.8               192        3000
## # i 1 more variable: sex <chr>
```

```
x |> ungroup()
```

```
## # A tibble: 33 x 7
##    species   island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>     <chr>             <dbl>         <dbl>             <dbl>       <dbl>
##  1 Adelie    Torgers~           38.8          17.6               191        3275
##  2 Gentoo    Biscoe             45.7          13.9               214        4400
##  3 Chinstrap Dream              45.7          17.3               193        3600
##  4 Adelie    Biscoe             39.6          17.7               186        3500
##  5 Adelie    Torgers~           34.4          18.4               184        3325
##  6 Adelie    Dream              37.3          16.8               192        3000
##  7 Adelie    Dream              35.7          18                 202        3550
##  8 Adelie    Biscoe             39            17.5               186        3550
##  9 Adelie    Dream              38.9          18.8               190        3600
## 10 Chinstrap Dream              42.4          17.3               181        3600
## # i 23 more rows
## # i 1 more variable: sex <chr>
```

Notice that the **Groups** attribute has been removed by `ungroup()`

Now that we've seen some examples of how to modify, and mutate, summarize, and group datasets. Let's move onto plotting the dataset.

---

slice_ Functions - extract specific rows from a grouping

```
df |> slice_head(n = 1) takes the first row from each group.
df |> slice_tail(n = 1) takes the last row in each group.
df |> slice_min(x, n = 1) takes the row with the smallest value of column x.
df |> slice_max(x, n = 1) takes the row with the largest value of column x.
df |> slice_sample(n = 1) takes one random row.
```

The `slice_` function have an alternative argument- instead of including `n=` for a specific number of rows, you can use `prop=<fraction>` where it'll return the specified fraction of rows.

```r
penguins |> slice_max(body_mass_g, n=3)
```

```
## # A tibble: 4 x 7
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <chr>   <chr>           <dbl>         <dbl>             <dbl>       <dbl>
## 1 Gentoo  Biscoe           49.2          15.2               221        6300
## 2 Gentoo  Biscoe           59.6          17                 230        6050
## 3 Gentoo  Biscoe           51.1          16.3               220        6000
## 4 Gentoo  Biscoe           48.8          16.2               222        6000
## # i 1 more variable: sex <chr>
```

```r
penguins |> slice_sample(prop=0.03)
```

```
## # A tibble: 10 x 7
##    species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##    <chr>   <chr>              <dbl>         <dbl>             <dbl>       <dbl>
##  1 Gentoo  Biscoe              45.2          14.8               212        5200
##  2 Adelie  Torgersen           45.8          18.9               197        4150
##  3 Adelie  Torgersen           37.8          17.1               186        3300
##  4 Gentoo  Biscoe              49.2          15.2               221        6300
##  5 Gentoo  Biscoe              42            13.5               210        4150
##  6 Adelie  Dream               42.3          21.2               191        4150
##  7 Gentoo  Biscoe              49.6          16                 225        5700
##  8 Adelie  Dream               39.2          18.6               190        4250
##  9 Adelie  Torgersen           36.7          18.8               187        3800
## 10 Adelie  Biscoe              41.4          18.6               191        3700
## # i 1 more variable: sex <chr>
```

Combining/Merging Tables

Column-wise Merging Tables

Remember, we saw `cbind()` used to combine to tables by columns. **dplyr** provides some additional functions that give you fine control on how the columns are combined.

Function	Description
`bind_cols()`	Paste/Joins columns into a single data frame. Rows must align
`left_join(x,y, by=...)`	Joins matching values in Y to X
`right_join(x,y, by=...)`	Joins matching values in X to Y
`inner_join(x,y, by=...)`	Joins data - but only retains rows that match between x and y
`full_join(x,y, by=...)`	Joins data and retrains all the rows regardless if they were in both.

Parameters/Arguments:

- `by=...` : To specify one or more columns to match based on the same name provide a vector of column names c("col1", "col2")
- `by=...` : To specify one or more columns to match based on different column names provide a named vector. c("col1"="V1", "col2"="V2")
- `suffix=...` : If columns of same names are present in both data frames, provide a list of suffix to use in the combined table. c("_x", "_y") This will add the suffix to the column of same names, so you can tell where they came from.

```
#untar('clinical.project-tcga-brca.2025-05-06.tar.gz', files='pathology_detail.tsv')
(pathology = read_tsv("pathology_detail.tsv", na=c("", "NA", "'--")))
```

```
## Rows: 1096 Columns: 86
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr  (8): project.project_id, cases.case_id, cases.submitter_id, diagnoses.d...
## dbl  (2): pathology_details.lymph_nodes_positive, pathology_details.lymph_no...
## lgl (76): pathology_details.additional_pathology_findings, pathology_details...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## # A tibble: 1,096 x 86
##    project.project_id cases.case_id    cases.submitter_id diagnoses.diagnosis_id
##    <chr>              <chr>            <chr>              <chr>
##  1 TCGA-BRCA          001cef41-ff86-4~ TCGA-E2-A1IU       b881807c-67e1-5a31-80~
##  2 TCGA-BRCA          0045349c-69d9-4~ TCGA-A1-A0SB       4f9c5557-83bf-50dc-b8~
##  3 TCGA-BRCA          00807dae-9f4a-4~ TCGA-A2-A04W       3ba22ba1-933f-5edf-b2~
##  4 TCGA-BRCA          00a2d166-78c9-4~ TCGA-AN-A0AM       d4a178f0-22d8-5bd4-95~
##  5 TCGA-BRCA          00b11ca8-8540-4~ TCGA-LL-A440       426cbd54-8cd7-5c29-b7~
##  6 TCGA-BRCA          011b9b2d-ebe5-4~ TCGA-A7-A26E       9ac74042-d755-5568-b3~
##  7 TCGA-BRCA          01263518-5f7c-4~ TCGA-A8-A07W       36a3ab37-6099-58e7-b3~
##  8 TCGA-BRCA          0130d616-885e-4~ TCGA-D8-A1XY       05b37f74-473c-58fd-ad~
##  9 TCGA-BRCA          01674b2c-5cf2-4~ TCGA-EW-A1P7       5de77adb-0f3b-5e4a-b9~
## 10 TCGA-BRCA          016caf42-4e19-4~ TCGA-AO-A128       e662f8ff-0f85-5d87-a6~
## # i 1,086 more rows
## # i 82 more variables: diagnoses.submitter_id <chr>,
## #   pathology_details.additional_pathology_findings <lgl>,
## #   pathology_details.anaplasia_present <lgl>,
## #   pathology_details.anaplasia_present_type <lgl>,
## #   pathology_details.bone_marrow_malignant_cells <lgl>,
## #   pathology_details.breslow_thickness <lgl>, ...
```

```
(bind_cols(clinical_uniq, pathology))
```

```
(combined = left_join(clinical_subset, pathology, by=c('cases.case_id'), suffix=c("", "_patho")))
```

```
## # A tibble: 1,098 x 116
##    cases.case_id cases.consent_type cases.days_to_consent cases.days_to_lost_t~1
##    <chr>         <chr>                             <dbl> <lgl>
##  1 001cef41-ff8~ Informed Consent                    -34 NA
##  2 0045349c-69d~ Informed Consent                     76 NA
##  3 00807dae-9f4~ Informed Consent                     19 NA
##  4 00a2d166-78c~ Informed Consent                      0 NA
##  5 00b11ca8-854~ Informed Consent                     11 NA
##  6 011b9b2d-ebe~ Informed Consent                     16 NA
```

```
##  7 01263518-5f7~ Informed Consent                      31 NA
##  8 0130d616-885~ Informed Consent                      36 NA
##  9 01674b2c-5cf~ Informed Consent                      13 NA
## 10 016caf42-4e1~ Informed Consent                       5 NA
## # i 1,088 more rows
## # i abbreviated name: 1: cases.days_to_lost_to_followup
## # i 112 more variables: cases.disease_type <chr>, cases.index_date <chr>,
## #   cases.lost_to_followup <chr>, cases.primary_site <chr>,
## #   cases.submitter_id <chr>, demographic.age_at_index <dbl>,
## #   demographic.age_is_obfuscated <lgl>, demographic.cause_of_death <lgl>,
## #   demographic.cause_of_death_source <lgl>, ...
```

```r
names(combined)
```

```
##   [1] "cases.case_id"
##   [2] "cases.consent_type"
##   [3] "cases.days_to_consent"
##   [4] "cases.days_to_lost_to_followup"
##   [5] "cases.disease_type"
##   [6] "cases.index_date"
##   [7] "cases.lost_to_followup"
##   [8] "cases.primary_site"
##   [9] "cases.submitter_id"
##  [10] "demographic.age_at_index"
##  [11] "demographic.age_is_obfuscated"
##  [12] "demographic.cause_of_death"
##  [13] "demographic.cause_of_death_source"
##  [14] "demographic.country_of_birth"
##  [15] "demographic.country_of_residence_at_enrollment"
##  [16] "demographic.days_to_birth"
##  [17] "demographic.days_to_death"
##  [18] "demographic.demographic_id"
##  [19] "demographic.education_level"
##  [20] "demographic.ethnicity"
##  [21] "demographic.gender"
##  [22] "demographic.marital_status"
##  [23] "demographic.occupation_duration_years"
##  [24] "demographic.population_group"
##  [25] "demographic.premature_at_birth"
##  [26] "demographic.race"
##  [27] "demographic.submitter_id"
##  [28] "demographic.vital_status"
##  [29] "demographic.weeks_gestation_at_birth"
##  [30] "demographic.year_of_birth"
##  [31] "demographic.year_of_death"
##  [32] "project.project_id"
##  [33] "cases.submitter_id_patho"
##  [34] "diagnoses.diagnosis_id"
##  [35] "diagnoses.submitter_id"
##  [36] "pathology_details.additional_pathology_findings"
##  [37] "pathology_details.anaplasia_present"
##  [38] "pathology_details.anaplasia_present_type"
##  [39] "pathology_details.bone_marrow_malignant_cells"
##  [40] "pathology_details.breslow_thickness"
##  [41] "pathology_details.circumferential_resection_margin"
```

```
##  [42] "pathology_details.columnar_mucosa_present"
##  [43] "pathology_details.consistent_pathology_review"
##  [44] "pathology_details.days_to_pathology_detail"
##  [45] "pathology_details.dysplasia_degree"
##  [46] "pathology_details.dysplasia_type"
##  [47] "pathology_details.epithelioid_cell_percent"
##  [48] "pathology_details.epithelioid_cell_percent_range"
##  [49] "pathology_details.extracapsular_extension"
##  [50] "pathology_details.extracapsular_extension_present"
##  [51] "pathology_details.extranodal_extension"
##  [52] "pathology_details.extraocular_nodule_size"
##  [53] "pathology_details.extrascleral_extension"
##  [54] "pathology_details.extrascleral_extension_present"
##  [55] "pathology_details.extrathyroid_extension"
##  [56] "pathology_details.greatest_tumor_dimension"
##  [57] "pathology_details.gross_tumor_weight"
##  [58] "pathology_details.histologic_progression_type"
##  [59] "pathology_details.intratubular_germ_cell_neoplasia_present"
##  [60] "pathology_details.largest_extrapelvic_peritoneal_focus"
##  [61] "pathology_details.lymph_node_dissection_method"
##  [62] "pathology_details.lymph_node_dissection_site"
##  [63] "pathology_details.lymph_node_involved_site"
##  [64] "pathology_details.lymph_node_involvement"
##  [65] "pathology_details.lymph_nodes_positive"
##  [66] "pathology_details.lymph_nodes_removed"
##  [67] "pathology_details.lymph_nodes_tested"
##  [68] "pathology_details.lymphatic_invasion_present"
##  [69] "pathology_details.margin_status"
##  [70] "pathology_details.measurement_type"
##  [71] "pathology_details.measurement_unit"
##  [72] "pathology_details.metaplasia_present"
##  [73] "pathology_details.micrometastasis_present"
##  [74] "pathology_details.morphologic_architectural_pattern"
##  [75] "pathology_details.necrosis_percent"
##  [76] "pathology_details.necrosis_present"
##  [77] "pathology_details.non_nodal_regional_disease"
##  [78] "pathology_details.non_nodal_tumor_deposits"
##  [79] "pathology_details.number_proliferating_cells"
##  [80] "pathology_details.pathology_detail_id"
##  [81] "pathology_details.percent_tumor_invasion"
##  [82] "pathology_details.percent_tumor_nuclei"
##  [83] "pathology_details.perineural_invasion_present"
##  [84] "pathology_details.peripancreatic_lymph_nodes_positive"
##  [85] "pathology_details.peripancreatic_lymph_nodes_tested"
##  [86] "pathology_details.prcc_type"
##  [87] "pathology_details.prostatic_chips_positive_count"
##  [88] "pathology_details.prostatic_chips_total_count"
##  [89] "pathology_details.prostatic_involvement_percent"
##  [90] "pathology_details.residual_tumor"
##  [91] "pathology_details.residual_tumor_measurement"
##  [92] "pathology_details.rhabdoid_percent"
##  [93] "pathology_details.rhabdoid_present"
##  [94] "pathology_details.sarcomatoid_percent"
##  [95] "pathology_details.sarcomatoid_present"
```

```
##  [96] "pathology_details.size_extraocular_nodule"
##  [97] "pathology_details.spindle_cell_percent"
##  [98] "pathology_details.spindle_cell_percent_range"
##  [99] "pathology_details.submitter_id"
## [100] "pathology_details.timepoint_category"
## [101] "pathology_details.transglottic_extension"
## [102] "pathology_details.tumor_basal_diameter"
## [103] "pathology_details.tumor_burden"
## [104] "pathology_details.tumor_depth_descriptor"
## [105] "pathology_details.tumor_depth_measurement"
## [106] "pathology_details.tumor_infiltrating_lymphocytes"
## [107] "pathology_details.tumor_infiltrating_macrophages"
## [108] "pathology_details.tumor_largest_dimension_diameter"
## [109] "pathology_details.tumor_length_measurement"
## [110] "pathology_details.tumor_level_prostate"
## [111] "pathology_details.tumor_shape"
## [112] "pathology_details.tumor_thickness"
## [113] "pathology_details.tumor_width_measurement"
## [114] "pathology_details.vascular_invasion_present"
## [115] "pathology_details.vascular_invasion_type"
## [116] "pathology_details.zone_of_origin_prostate"
```

Row-wise Merging Tables

Remember, we saw `rbind()` used to combine to tables by rows. **dplyr** provides some additional functions that give you fine control on how the columns are combined.

Function	Description
bind_rows()	Paste/Joins rows from 2 tables into a single data frame. Columns must align
intersect(x,y,...)	Return rows that appear in both X and Y
setdiff(x,y,...)	Return rows in X but not in Y
union(x,y,...)	Joins X and Y but only unique rows (duplicate rows removed
union_all(x,y,...)	Joins X and Y duplicates are included
semi_join(x,y,by=...)	Returns rows of X that have a match in Y
anti_join(x,y, by=...)	Returns rows in X that don't have a match in Y